Memo: Standardise on python-pptx for Monthly Reporting Decks

To: Engineering Director
From: Reporting Engineering Team
Date: 4 November 2026
Decision requested: Approve python-pptx as the single library for monthly decks

Summary

We recommend standardising on python-pptx to generate our monthly reporting decks. Our team writes Python and our data pipelines already run in Python, so python-pptx lets the same people and the same code go from query to finished deck without a second language or a hand-off.

pptxgenjs is a capable library and in some areas more polished. It would, however, add Node.js tooling and a JavaScript skill set that the team does not have today. We judge that cost to outweigh its advantages for our use.

Context

Monthly reporting decks are currently produced with a mix of manual editing and ad hoc scripts. That makes the output inconsistent, slow to refresh and dependent on whoever last touched the process. We want one supported approach that is reviewable, testable and repeatable.

Comparison

Criterion	python-pptx	pptxgenjs
Language fit	Python; matches team skills and data stack	JavaScript/Node; new toolchain for the team
Data integration	Direct use of pandas, SQL clients and existing report code	Needs data exported to JSON or a Node data layer
Templates	Opens an existing .pptx template and fills its layouts and placeholders	Mostly builds slides in code; template reuse is more limited
Charts and tables	Native charts and tables; fine control sometimes needs XML edits	Rich chart and table options with a friendly API
Testing and CI	Fits our existing pytest and CI setup	Requires a parallel Node test and build setup
Main weakness	No rendering engine; animations and some effects unsupported	Second language to hire for, train on and maintain

Recommendation

Adopt python-pptx as the standard for all monthly reporting decks, built on one approved corporate template. Wrap it in a small internal module so slide types such as title, KPI summary, chart and table are defined once and reused. Do not start new pptxgenjs work; migrate any existing pptxgenjs decks only when they are next changed.

Risks and mitigations

- **Feature gaps.** python-pptx does not cover every PowerPoint feature, such as animations. Mitigation: agree a limited set of slide types up front and use XML-level edits only for the rare exception.
- **Visual drift.** Generated slides can differ subtly from the template. Mitigation: render decks to images in CI and review them against reference slides.
- **Library maintenance.** Release pace of an open-source dependency is outside our control. Mitigation: pin versions and keep the wrapper module thin so a replacement stays feasible.
- **Single-owner knowledge.** The wrapper could end up understood by one person. Mitigation: document it and have two engineers review every change.
- **Reporting-owner pushback.** Stakeholders may expect design freedom that code templates restrict. Mitigation: involve them in choosing the template and slide types.

Next steps

Action	Owner	Target
Approve the recommendation	Engineering Director	Within one week
Confirm corporate template and slide types with report owners	Reporting Product Owner	Two weeks
Build the wrapper module and a pilot deck from last month's data	Tech Lead, Reporting Engineering	Four weeks
Add CI step that renders decks and checks them against references	Platform Engineer	Five weeks
Review pilot and decide whether to retire manual deck production	Engineering Director	Six weeks